

Automatic Classifiers for Formative Assessment

Wassim Mahfouz
Faculty of Computer Science and Automation
Ilmenau University of Technology
Ilmenau, Germany
wassim.mahfouz@tu-ilmenau.de

Heinz-Dietrich Wuttke
Faculty of Computer Science and Automation
Ilmenau University of Technology
Ilmenau, Germany
dieter.wuttke@tu-ilmenau.de

Abstract— This Research to Practice Full Paper is driven by the question: How can a teacher of a large class be effectively supported for formative assessment? The paper suggests a framework for automatic classifiers / predictive models and its data integration tool for teachers. To design an input data set for automatic classifier, the tool enables the teachers to integrate data extracted from paper-based exams; computer assisted formative assessments and LMSs, with learning disposition data, collected by applying the Buckingham Shum and Deakin Crick's theoretical framework for dispositional learning analytics. The suggested framework and its tool are tested with the real assessment data of 129 students collected during conducting the Computer Organization (hereafter CO) course. The results of this CO test scenario show how teachers can interpret the outcomes of the automatic classifiers as decision-support recommendations to improve the planning for formative assessment. As an example, it is presented and discussed how the CO teacher can improve his/her strategies in formative assessment for different students groups (at-risk students, medium students, good students and excellent students). The paper shows also how reports for classifiers accuracy comparison can be produced and understood by teachers. Related works are discussed to show the differences and benefits of the presented framework. Main advantages are: the possibility to use its automatic classification algorithms instead of the statistical regression algorithms and the possibility to use its integration tool to integrate data collected from applying a theoretical framework with data from e-learning /e-assessment as input data sets for automatic classifiers.

Keywords— *assessment in engineering education, planning and decision support for formative assessment, automatic classification.*

I. INTRODUCTION

Assessment is an essential component of learning and teaching, as it allows the quality of both teaching and learning to be judged and improved [1]. It often determines the priorities of education [2], it always influences practices on learning [3]. Changes in curricula and learning objectives are ineffective if assessment practices remain the same [4], as learning and teaching tend to be modelled against the assessment purposes and goals. Assessment is usually understood to have two purposes: summative and formative. Summative assessment is used to judge students' achievements at the end of an instructional unit, whereas formative assessment aims to gather evidence about students' learning progress to influence teaching methods and priorities [2]. Although researchers have long advocated the potentially positive influences of formative assessment on learning, its implementation has been challenging. One challenge is the

alignment of formative and summative practices during the course. Another complicating factor is that high workload of the teacher, due to the big class size, leads to low frequencies of formative assessment practices and their accompanying feedbacks. Consider for example the first year CO class that has 129 engineering students. Can the CO teacher be effectively supported for formative assessment? How? In this paper, we will show how our suggested automatic classifiers can support the CO teacher. In our previous and current research projects [5, 6, 7] we have investigated how technology-enhanced formative assessment tools can support a teacher and how they fit the educational goals and skills needed for the 21st century in the engineering education domain. In [6], we develop a mobile assessment app [6] to help us understand how we can design inquiry-based practical teaching goals and their formative assessment practices in remote and online labs. In this paper our **main contributions** are:

- We have built a framework for automatic classifiers/predictive models to enable teachers to predict student performances (Sections 4 and 5). Its innovative features are :
 - Reports for automatic classification (i.e. prediction), and automatic classifiers accuracy comparison for decision support in formative assessment can be produced and easily interpreted by teachers. (Section 5).
 - Through its help documentation the teachers can learn more about how to effectively use its automatic classification algorithms (Section 4).
- We have built a teacher support tool to integrate and customize input data sets for automatic classifiers (Section 2 and 3). Its innovative features are:
 - It is designed to enable teachers integrate data collected from applying a theoretical framework “E.g. The Buckingham Shum and Deakin’s framework [8].” with data from several e-learning/e-assessments systems.
 - It is scalable. Teachers in disadvantaged schools/universities (i.e. Schools/universities that have no support for e-learning / e-assessment systems) can also use it (Section 3).

In Section 6, the related works and in Section 7, the conclusions and future work will be presented. In the next section, we present the teacher support tool to integrate and customize input data sets for automatic classifiers.

II. TEACHER SUPPORT TOOL TO CUSTOMIZE INPUT DATASETS FOR AUTOMATIC CLASSIFIERS

The tool, shown in Fig. 1 helps the teacher to integrate the data, collected from different sources. The sources have three types: technology enhanced sources, traditional sources, and self-reports sources for dispositional learning. The technology enhanced sources can be the databases of different e-learning systems (e.g. the TU-Ilmenau moodle-system [5], an e-assessment system (e.g. the *askme!* system [7]), and data, collected from students using our mobile assessment app called TEASE [6]). In addition, the use of traditional sources like paper exam results and self-report surveys is possible. In the context of the CO course, we apply the proposal of Buckingham Shum and Deakin Crick [8] for dispositional learning analytics. They propose an infrastructure that combines learning activity generated data with learning dispositions, values and attitudes measured through self-report surveys, which are fed back to students and teachers through visual data analytics.

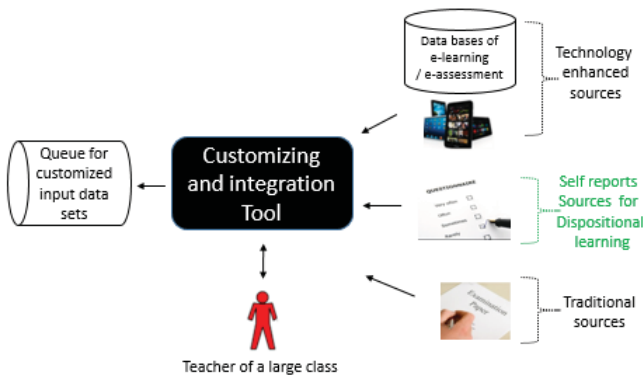


Fig. 1. Teacher support tool to customize input datasets for automatic classifiers

A teacher should use the tool in two steps. She/he starts with the data integration step, followed by the data customization step. In this paper, we do not cover in detail the integration step. In brief, this step enables the teacher to collect data from different sources and output them in an integrated data set. This set then **can be customized as predictors and predicted variables for predictive models (i.e. automatic classifiers)**. The teacher can customize many data sets and output them in a queue as seen in Fig. 1. The queue works as an input buffer to the automatic classifiers framework (presented in section 4). The next section shows an example for customizing a demo dataset that can be achieved not only by the CO teacher but also by teachers in other education domains.

III. CUSTOMIZING A DATASET AS DEMO FOR TEACHERS IN DIFFERENT EDUCATION DOMAINS

All teachers of a large class in different education domains have traditional data sources (i.e. paper exams results) but some of the teachers (E.g. teachers in schools/universities that have no support for e-learning / e-assessment systems) do not have the technology-enhanced sources, and self-reports sources in Fig. 1. Therefore we would prefer to select for this demo only the CO paper exam results to show how our **tool is simple and scalable and it can be also used by teachers in schools/universities that have no support for e-learning / e-assessment systems**. In brief, the CO paper exam results represent the summative exam scores of 129 students. The exam data was collected during conducting the CO course in

last winter semester. This summative exam consists of nine exam tasks and each task covers a topic that is subject to learn by the students during two weeks of time. We reorder the scores of the nine exam tasks in a way that the first exam task covers approximately the CO lectures' content of the first two weeks; the second exam task covers the lectures' content of the following 2 weeks and so on. Thus, we get a fictive timeline with nine time points for formative assessment practices. We call these 9 time points {week_2, week_4,..., week_18} and the matching exam task scores for them are called predictive variables or simply predictors if we use the terms of a machine learning community. In contrast to predictive variables that work as inputs for a predictive model (i.e. the automatic classifier), the predictive model has an output variable or a predicted variable. For example, the student final exam score can be a predicted variable for our demo dataset.

The customizing tool has many support user interfaces (UIs), as seen in Fig. 2 and Fig. 4, to enable teachers adjusting and customizing the predictive and predicted variables over the time line that fits his/her prediction tasks. As an example, we will customize two data sets for two prediction tasks of the CO teacher. The prediction tasks are:

- **Prediction task (1):** The teacher is interested in predicting the final exam score with 2 classes (Fail, Success) with including the input predictors of all 2 weeks based periods (2, 4, 6, 8,10, ..., 16, and 18).
- **Prediction task (2):** The teacher is interested to predicting the final exam score with 4 classes (Fail, Medium, Good, Excellent) with including only the input predictors of the periods (2, 4, 6, 8 and 10).

For prediction task (1), Fig. 2 shows the 9 input variables or predictors customized by the teacher to fit the 9 time points i.e. {week_2, week_4,..., week_18}. It shows also the three variables student_id, study_program and final_exam_score.

- student_id represents the identifiers of the 129 students
- The study_program represents to what program the students belong. There are four programs {BT (Biomedical Engineering), II (Informatics Engineering), MA (Mathematics), IN (Informatics)}.
- final_exam_score: the sum of all exam task sub-scores.

In Fig. 2, the final_exam_score variable was customized by the CO teacher to produce the next output variable:

- output_predicted_final_exam_score_labeled_by_2_classes.

This output predicted variable was produced through applying a classification schema defined by the teacher. The definition for a schema with two classes {Success, Fail} can be as follows: if a student gets more than 22.5 out of 50, he/she belongs to the "Success"-class, otherwise he/she belongs to "Fail"-class. The teacher can get the distribution in Fig. 3. This distribution is a visual representation for the final_exam_score variable after applying the two classes (Success, Fail).

Creating a classification schema and applying it on a variable is also one of the support features, provided by the customizing tool. This feature can enable the teacher to divide the students' scores (the variables) into qualified classes to help him/her to understand and differentiate the students' difficulties. For example, to differentiate the levels of the

success, Fig. 4 shows how the teacher has used the three subclasses (Medium, Good, and Excellent) through applying a classification schema with 4 classes (Fail, Medium, Good, Excellent) instead of two classes (Fail, Success).

No.	Name
1	student_id
2	study_program
3	final_exam_score
4	input_week_2_predictor
5	input_week_4_predictor
6	input_week_6_predictor
7	input_week_8_predictor
8	input_week_10_predictor
9	input_week_12_predictor
10	input_week_14_predictor
11	input_week_16_predictor
12	input_week_18_predictor
13	output_predicted_final_exam_score_labeled_by_2_classes

Fig. 2. Customized data set for prediction task (1)

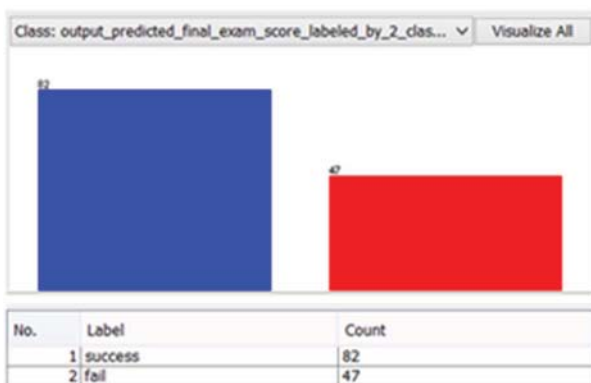


Fig. 3. Distribution for final_exam_score variable after applying the two classes (Success, Fail)

Fig. 5 shows the distribution of the four classes. The customization Tool enables also the teacher to filter the input predictor variables to fit the requirements of his/her prediction task. For example for the prediction Task (2), Fig. 4 shows the predictors of the 2 weeks based periods (2, 4, 6, 8 and 10). The other predictors of the periods (12, 14, 16, and 18) were filtered because the CO teacher wants to predict early (i.e. before week 12) the four classes (Fail, Medium, Good, Excellent).

No.	Name
1	student_id
2	study_program
3	final_exam_score
4	input_week_2_predictor
5	input_week_4_predictor
6	input_week_6_predictor
7	input_week_8_predictor
8	input_week_10_predictor
9	☒ output_predicted_final_exam_score_labeled_by_4_classes

Fig. 4. Customized data set for the prediction task (2)

To sum up, the customized data sets shown in Fig. 2 and Fig. 4, are examples to produce predictors and predicted

variables for predictive models (i.e. automatic classifiers). These variables are pushed as first input (input 1 in Fig. 6) to the automatic classifiers framework. The framework we describe in the next section.



Fig.5. Distribution for final_exam_score variable after applying the 4 classes (Excellent, Good, Medium, Fail)

IV. AUTOMATIC CLASSIFIERS FRAMEWORK

The framework, shown in Fig. 6 has two inputs and one output beside its three components. The components are: 1) a factory for classification or predictive algorithms: the factory offers software infrastructure to create, initialize and run the algorithms. 2) The tuning interfaces and 3) the reports designing tool. The reports designing tool works as a single interactive window for the teacher. It uses the tuning interfaces to build friendly interactive interfaces to enable the teacher tune 2 inputs: Input 1 and Input 2. Input 1 is the predictors and predicted variables fetched by the designing tool from the queue (explained in section 2). Input 2 is a template of an algorithm. The teacher can select it from the repository of the algorithm templates. The selected template defines the algorithm, and its configuration parameters. After tuning the two inputs, the teacher can run the selected algorithm to get a model for automatic classifiers or for predictions. The designing tool presents this model to enable the teacher to adjust and output it for two types of reports.

- The first is a report for recommendations and decision support to improve planning for formative assessment.
- The second is a report for classifiers accuracy comparison (see Fig. 6).

The framework offers classification algorithms depending on two java open source projects: The first is WEKA (Waikato Environment for Knowledge Analysis) workbench [9] and the second is DeepLearning4j (Deep Learning for Java) [10]. The WEKA based classification algorithms include the following techniques:

- Logistic Regression, Naive Bayes, Decision Trees, k-Nearest Neighbors, Support Vector Machines, and Neural networks.

Beside the WEKA based traditional neural network algorithms, the framework offers also more advanced algorithms for deep learning in neural networks [10].

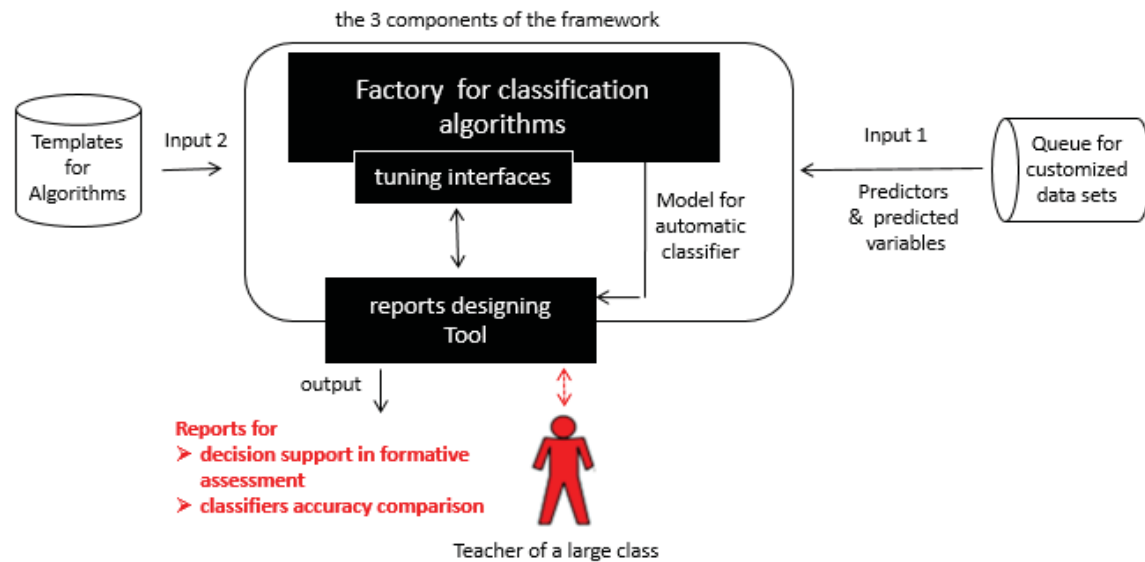


Fig 6. The automatic classifiers framework

To tune and configure the algorithms, the framework has two kinds of tuning interfaces: 1) Simple tuning interface for teachers and 2) more advanced technical tuning interfaces for the administrator of the framework. Fig. 7 shows an example how the CO teacher can configure the minimum number of the feature student objects in a leaf for the C4.5 algorithm [11] (The C4.5 is called J48 in WEKA community [9]).



Fig 7. Teacher UI to configure the minimum number of the feature student objects in each leaf for the J48 algorithm

This value "10" means that each leaf in the decision or classification tree produced by applying the J48 algorithm should contain at minimum 10 feature student objects. Feature student object means that each feature object represents a student. For advanced configuration of the J48 algorithm.

Fig. 8 shows how the administrator can adjust its control parameters.

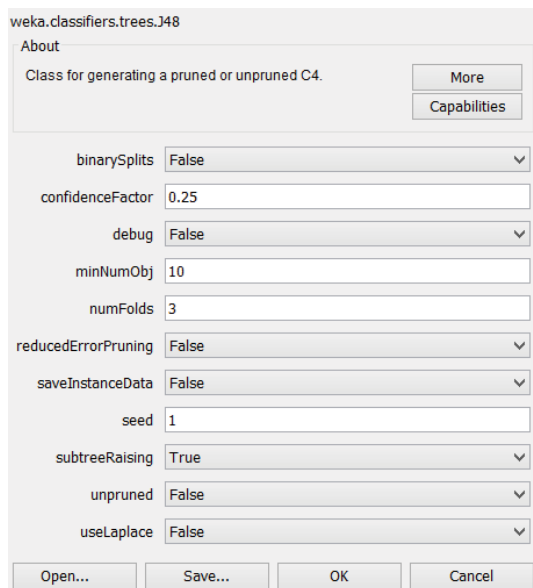


Fig 8. Administrator UI for control parameters of J48 algorithm

Most of the control parameters of the framework's algorithms that are of minor interest by the teacher or the administrator are configured by default values that grantee the best prediction performance of the selected algorithm. Those default values are defined and suggested by machine learning communities. For example, Fig. 9 shows how default values are selected to train the J48 algorithm. The Weka community has found that the training technique cross-validation with 10 folds leads to best prediction accuracy of the J48 algorithm. Instead of the suggested default values, Fig. 9 shows other options for the teacher or the administrator to divide the input data set into training and test data sets. The options are:

- a training set can be used separately,
- the test set can be supplied by an external file,
- the cross-validation with folds can be used for test and training sets or finally
- the training and test sets can be defined via percentage.

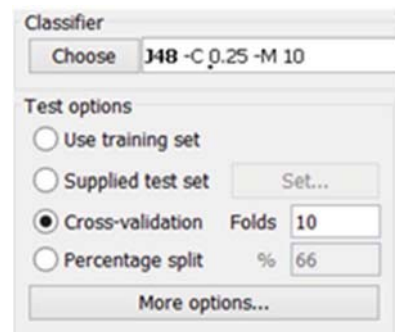


Fig 9. Administrator UI to configure the training and test sets for J48 algorithm

In order to increase its usability, the framework provides also help documentations for teachers to explain its algorithms and the main **problem of performance prediction of a student** addressed by these algorithms. The formal definition of the problem is explained in the documentations [9] as follows.

Given n training samples $(X_1, z_1), (X_2, z_2) \dots (X_n, z_n)$, where

- X_i is the feature vector or object for student a_i , $X_i \in R^m$ where m is the total number of features.
- A is the set of n students where $A = \{a_1, a_2, a_3 \dots a_n\}$.
- z_i is the student performance status for example (fail, success), $z_i \in \{-1, +1\}$.

To predict the performance of a student, the following prediction function is proposed:

$$z = F(A/X)$$

Where,

$$F(A/X) = \begin{cases} \geq 0 & \text{if } z = +1, \text{ success} \\ < 0 & \text{if } z = -1, \text{ fail} \end{cases}$$

Prediction algorithm Task: The goal is to learn a predictive function $\hat{F}(\cdot)$ or alternatively, to predict whether a student will have fail or success in final exam. It is written as:

$$\hat{z} = \hat{F}(A/X)$$

To evaluate an algorithm's prediction performance, the framework's documentations explain also four standard evaluation metrics (precision, recall, accuracy and F1-score) [9]. In brief, these metrics are defined as:

$$\text{Recall} = \frac{tp}{tp + fn}$$

$$\text{Precision} = \frac{tp}{tp + fp}$$

$$\text{Accuracy} = \frac{tp + tn}{tp + tn + fp + fn}$$

Where

- tp = true positives: number of instances predicted positive that are actually positive
- fp = false positives: number of instances predicted positive that are actually negative
- tn = true negatives: number of instances predicted negative that are actually negative
- fn = false negatives: number of instances predicted negative that are actually positive

The traditional F-measure or balanced F-score (F1 score) is the harmonic mean of precision and recall.

$$F_1 = 2 \cdot \frac{1}{\frac{1}{\text{recall}} + \frac{1}{\text{precision}}} = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

The framework uses the last measures or metrics to provide teachers by the classifiers accuracy comparison reports. Beside these comparison reports, the framework provides the outcomes of applying its classification algorithms as reports for decision support in formative assessment. Next section presents the decision support reports and the classifiers accuracy comparison reports for some automatic classifiers applied on the two customized datasets (see Fig. 2 and Fig. 4) for the two CO teacher's prediction tasks described in section 3.

V. REPORTS OF APPLYING AUTOMATIC CLASSIFIERS BY THE CO TEACHER

In this section, we present first the reports related to the CO teacher's prediction task (1) and then the reports related to the CO teacher's prediction task (2).

A. Reports Related to the Prediction Task (1)

Fig. 2 shows the customized data set for prediction task (1). In this prediction task the teacher wants to predict the final exam score with two classes (Fail, Success) by using the input predictors of all 2 weeks based periods (week_2_predictor, week_4_predictor..., week_18_predictor). Fig. 10 shows the classification or prediction tree of applying the J48 algorithm on the customized data set for this prediction task. To configure the variable "minimum number of feature student objects in a leaf", see Fig. 7, the teacher selects the value 10. The teacher runs the J48 algorithm again with value 6 instead of 10. For the value 6, he gets the outcome classification or prediction tree seen in Fig. 11. Each node of the trees in Fig. 10 and Fig. 11 is self-described. Through clicking on it, the teacher can get more information about the topic planned to be taught in a period of two weeks and about the max exam score for this topic. Fig. 10 shows an example of clicking on the tree node "week_10_predictor".

The tree leafs and their labels in Fig. 10 can be interpreted as follows:

- Given that the week_10_predictor variable is the students' scores in the simulated formative assessment practice conducted in week 10, and the week_16_predictor variable is the students' scores in the simulated formative assessment practice conducted in week 16.
- For the leaf with label "**Fail (45 / 7)**", the J48 algorithm predicts that if week_10_predictor ≤ 3 then there are 45 students classified in "fail" class (i.e. they will fail the final exam). 7 students out of the 45 students were wrongly classified in this "fail" class.
- For the leaf with label "**Success (71 / 2)**", the J48 algorithm predicts that if week_10_predictor > 3 and week_16_predictor > 1 then there are 71 students are classified in "success" class (i.e. they will pass the final exam). Two students out of the 71 students were wrongly classified in this "success" class.
- For the leaf with label "**Fail (13 / 6)**" The J48 algorithm predicts that if week_10_predictor > 3 and week_16_predictor < 1 then there are 13 students are classified in "fail" class. 6 students out of the 13 students were wrongly classified in this "fail" class.

In the same way the tree leafs in Fig. 11 can be easily interpreted.

Now we can discuss the planning for formative assessment (at-risk student strategy)

The decision trees in Fig. 10 and Fig. 11 are examples to support the CO teacher take decisions for his/her at-risk strategy in formative assessment. Leafs in these trees, labeled with "fail", are the target groups of this at-risk student strategy.

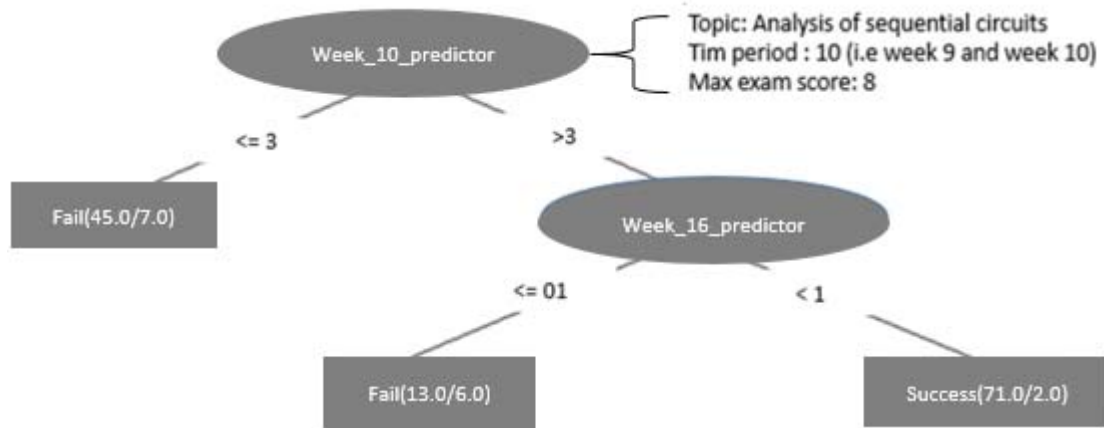


Fig 10: Decision or classification tree for J48 algorithm: **minimum number of students in a leaf=10**

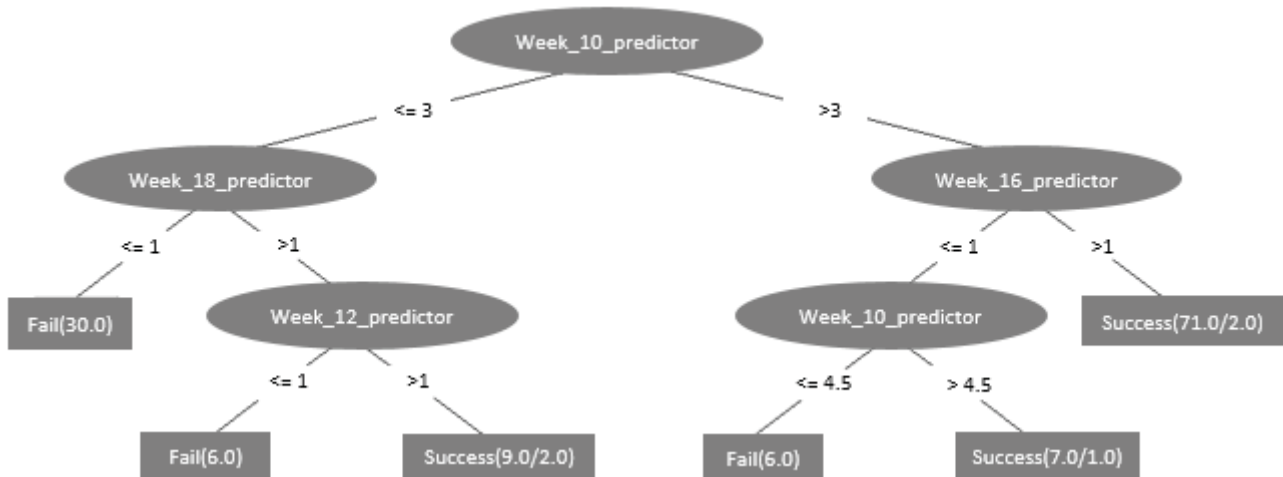


Fig 11: Decision or classification tree for J48 algorithm: **minimum number of students in a leaf=6**

In Fig. 10 there are two at-risk student groups

- At-risk group (Label “Fail (45 / 7)”): Students in this group will fail the final exam, because they have less than 3 out of 8 for the exam task related to the topic “**Analysis of sequential circuits**” that takes places in the time period 10 (i.e. week 9 and 10). That means the teacher has 8 weeks before the final exam because the final exam will take place in the end of the time period 18 (i.e. week 17 and 18). During these 8 weeks, the teacher can plan teaching offers related to the week 10 topic “**Analysis of sequential circuits**” to support this at-risk student group.
- At-risk group (“fail (13 / 6)”: In this group the students have more than 3 out of 8 for the exam task related to the topic “**Analysis of sequential circuits**” but they will fail because they are weak in the Topic related to the time period 16 (week 15 and 16). That means the teacher has only 2 weeks before the final exam to support this at-risk student group.

The predictive power per CO topic for an input predictor variable can be also explained to the CO teacher. For example, in Fig. 10, the CO teacher can quickly understand why the predictive power of the variable week_10_predictor (that represents the Topic “**Analysis of sequential circuits**”) is so high. The value of this variable alone (when it is less than 3 out of 8) can lead to predict that about 45 students will fail the final exam and therefore they can be early classified in the “at-risk” student group depending on only one input predictor

variable (i.e. week_10_predictor). If the teacher is interested in deep understanding the difference of predictive powers of all input predictor variables, he can get from our framework a report for information gain (also called **entropy**). For example, the next table 0 shows the information gain (or entropy) Report for the input predictor variables in Fig. 2 defined by the CO teacher for his prediction task (1).

TABLE 0. INFORMATION GAIN (OR ENTROPY) REPORT FOR THE INPUT PREDICTORS VARIABLES IN FIG. 2

input predictor variable	information gain (or entropy)
week_10_predictor	0.438
week_8_predictor	0.325
week_12_predictor	0.303
week_14_predictor	0.27
week_18_predictor	0.238
week_16_predictor	0.233
week_6_predictor	0.215
week_4_predictor	0.199
week_2_predictor	0.145

The information gain (or entropy) in table 0 vary from 0 (no information) to 1 (maximum information). Those input predictor variables that contribute more for prediction algorithm than others (i.e. they have more predictive power) will have a higher information gain value (like the value **0.438** of the week_10_predictor) and can be selected by the teacher for his prediction tasks. Those variables that do not have much predictive power will have a lower score for entropy (like week_2_predictor in table 0) and therefore can be filtered or removed if the teacher is not interested in their low predictive

power. In Fig. 11, there are three at-risk groups (3 leafs with labels “Fail”). These groups maybe can fit the CO teacher better if he/she has more time and teaching resources for at-risk student strategy. The decision trees in Fig. 10 and Fig. 11 can be enhanced by other reports to help the teacher better understand the performance of the applied algorithm.

TABLE 1. PERFORMANCE REPORT OF J48 ALGORITHM USED TO GET THE DECISION TREE SHOWN IN FIG. 11

Test Mode (Folds)	Correctly classified instances	Incorrectly classified instances	Root mean squared error	Accuracy
15	110	19	0.3451	85.2713 %
10	113	16	0.3022	87.5969 %
5	111	18	0.328	86.0465 %

TABLE 1-1. DETAILED ACCURACY BY PREDICTED CLASS FOR PERFORMANCE REPORT IN TABLE 1.

Test Mode	Detailed accuracy by predicted class			
	Precision	Recall	F-Measure	Class
15 Folds	0.899	0.866	0.882	success
	0.78	0.83	0.804	Fail
10 Folds	0.923	0.878	0.9	success
	0.804	0.872	0.837	Fail
5 Folds	0.89	0.89	0.89	success
	0.809	0.809	0.809	fail

The reports of the algorithm-performance can be also designed by the teacher to cover the applying of the algorithm with different test modes. An example of a test mode, in machine learning terms, can be the cross-validation technique with its number of folds. With three values for folds (5, 10 and 15) table 1 shows a performance report for the J48 algorithm that has been applied by the CO teacher to get the decision tree shown in Fig. 11. Table 1-1 is added to show the detailed accuracy by a predicted class for performance report in table 1. To compare between algorithms for decision tree, the teacher can apply other algorithms like, AD Tree, REP Tree, BF Tree and simpleCart. Table 2 shows a performance report for the simpleCart algorithm [12] applied by the CO teacher with the same parameter configuration settings as used to apply the J48 algorithm. For example, the CO teacher for both algorithms (J48 and simpleCart) used the value 6 for the configuration variable “minimum number of students in a leaf”.

TABLE 2. PERFORMANCE REPORT OF SIMPLECART ALGORITHM USED TO GET THE DECISION TREE SHOWN IN FIG. 12

Test Mode (Folds)	Correctly classified instances	Incorrectly classified instances	Root mean squared error	Accuracy
15	111	18	0.3484	86.0465 %
10	110	19	0.3525	85.2713 %
5	109	20	0.3535	84.4961 %

TABLE 2-1. DETAILED ACCURACY BY PREDICTED CLASS FOR PERFORMANCE REPORT IN TABLE 2.

Test Mode	Detailed accuracy by predicted class			
	Precision	Recall	F-Measure	Class
15 Folds	0.872	0.915	0.893	success
	0.837	0.766	0.8	Fail
10 Folds	0.854	0.927	0.889	success
	0.85	0.723	0.782	Fail
5 Folds	0.852	0.915	0.882	Success
	0.829	0.723	0.773	Fail

Table 2-1 is added to show the detailed accuracy by a predicted class for performance report in table 2. In contrast to the decision/classification tree with six leafs produced by

the J48 in Fig. 11, applying the simpleCART algorithm produces only one tree node with 2 leafs. Fig. 12 shows this tree when the accuracy is maximum in table 2. That means for first row in table 2. Fig. 13 shows the accuracy comparison for the two automatic classifiers (J48 and simpleCart). If a teacher is interested in a more detailed accuracy of the classifiers, he can also get a detailed accuracy comparison reports by a predicted class using the metrics (precision, recall, and F-measure).



Fig 12. Decision or classification tree for simpleCART algorithm: minimum number of students in a leaf=6

As an example for the two automatic classifiers (J48 and simpleCart), Fig. 14 shows the classifier precision comparison for class “success”. Based on reports in Fig. 13 and Fig. 14 the CO teacher can easily understand that the J48 algorithm outperforms the simpleCART algorithm.

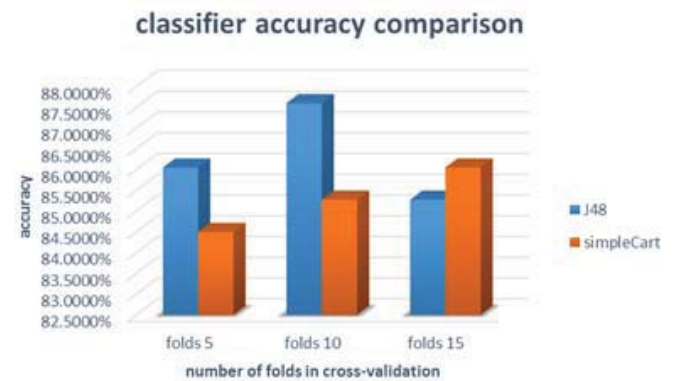


Fig 13. Report for classifiers accuracy comparison



Fig 14. Report: classifiers precision comparison for class “success”

B. Reports Related to the Prediction Task (2)

Beside at-risk student strategy, the planning for formative assessment in CO course can also include strategies for student who will pass the final exam with grades medium, good, or even excellent. For these kind of strategies, prediction task (2) is an example to show how the CO teacher can get decision tree. In prediction task (2), the CO teacher is interested to predict the final exam score with four classes (Fail, Medium, Good, Excellent). He wants to include only the input predictors of the time periods (2, 4, 6, 8 and 10) in order to predict early and have enough time (8 weeks) before the final exam for his/her formative assessment strategies.

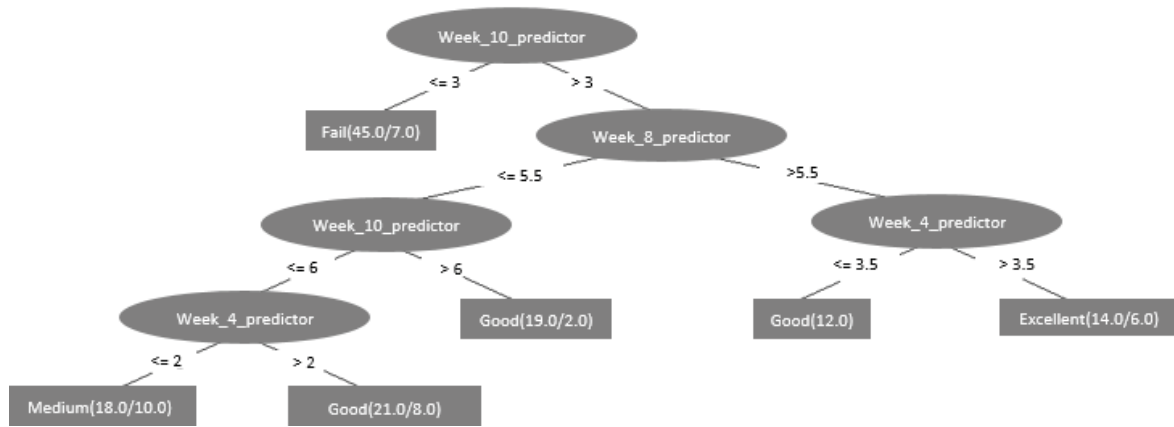


Fig 15: Decision or classification tree (J48 algorithm) for the 4 classes (Excellent, Good, Medium, Fail): minimum number of students in a leaf=6

Fig. 15 shows the decision tree of this prediction task. Here, the teacher can find

- One group for medium student (leaf with label “Medium”): planning of formative assessment practices for this group can be done via our offers in the e-learning Moodle system of TU-Ilmenau [5].
- Three groups for good students (three leafs with label “Good”): Offers to improve the motivation of those students can be virtual experiments in our remote lab [13].
- One group for excellent student (leaf with label “Excellent”). Offers to improve the motivation of those students can be practical experiments for extend models in our remote lab [13].

To sum up, in this section we have demonstrated how the CO teacher can produce reports for performances of classification algorithms, for classifiers accuracy comparison, and for decision support in formative assessment. How the CO teacher can improve his strategies in formative assessment for at-risk students group, medium students group, good students group and excellent students group were also discussed.

VI. RELATED WORKS

For predicting student performances, several works have been done by applying classification techniques. In his PhD thesis [14] Thai-Nghe shows how the student performances can be predicted in an intelligent tutoring system. Whereas Thai-Nghe’s work is limited to predict student performances inside an intelligent tutoring system, our work is different in providing the teacher by a support tool to integrate and customize input datasets for automatic classifiers. This tool can enable the teachers to predict inside the technology enhanced tutoring systems or outside of this system. To put it in other words, teachers in disadvantaged schools/universities who have no intelligent tutoring systems can also use the tool. It enables also the teachers to integrate data sources from inside the technology enhanced tutoring systems and outside these systems (E.g. data extracted from paper based exams and data collected by applying the Buckingham Shum and Deakin Crick’s theoretical framework for dispositional learning) to do the prediction tasks. In their inspiring work Dirk et al [15] use a leaner regression algorithm to present a predictive model/automatic classifier and to study its stability and sensitivity with a large input data set. Our automatic classifiers framework differs the work of Dirk et al [15] in providing not only regression algorithms but also many other data mining algorithms (Naive Bayes, Decision Trees, k-Nearest Neighbors, Support Vector Machines, and Neural networks).

In his PhD thesis [16], Huang has presented many statistical and data mining algorithms to solve the problem of predicting the student academic performance in an engineering dynamics course. Our work differs Huang’s work by providing the deep learning algorithms beside the data mining algorithms. Our framework’s deep learning algorithms (i.e. automatic feature extraction techniques) can help the CO teacher better to understand the unstructured data (i.e. usage student data) of the web based CO learning/ assessment materials in the e-learning/e-assessment systems and remote Laboratory of TU-Ilmenau. Beside the framework’s deep learning algorithms, our work differs Huang’s work by providing the integration tool. The data integration problem of e-learning/e-assessment systems is not addressed by Huang’s work.

VII. CONCLUSIONS AND FUTURE WORK

The automatic classifiers framework and its data integration tool for supporting teachers in planning formative assessment were presented. As an example to show how the tool can be scaled out and to be used by teachers in disadvantaged schools /universities (i.e. schools /universities that have no support for e-learning and e-assessment systems) the paper exams results of 129 CO students were integrated and customized to produce input predictor variables & output predicted variables for predictive models (i.e. automatic classifiers). We have demonstrated how the CO teacher can produce reports for performances of the framework’s classifiers algorithms, for classifiers accuracy comparison, and for decision support in formative assessment. How the CO teacher can improve his/her strategies in formative assessment for at-risk students group, medium students group, good students group and excellent students group were also presented and discussed. Predictive models based on other factors that contextualize the data may further improve the strength of the evidence. In our future work, we will present how the big databases of the CO course in the e-learning/e-assessment systems of TU-Ilmenau can be integrated with data collected by applying the Buckingham Shum and Deakin Crick’s theoretical framework for dispositional learning analytics to produce more efficient input predictor variables. More advanced automatic classifiers of the framework will be also presented in our future work. We will also present the framework’s deep learning algorithms (i.e. automatic feature extraction techniques) to help the CO teacher to better understand the unstructured data (i.e. usage student data) of the web based CO learning/ assessment materials in the e-learning/e-assessment systems and remote Laboratory of TU-Ilmenau.

ACKNOWLEDGMENT

The teaching team of the computer organization course at the TU- Ilmenau has supported the work, reported in this paper.

REFERENCES

- [1] Ferrari, A., Cachia, R., Punie, Y.: Innovation and Creativity in Education and Training in the EU Member States: Fostering Creative Learning and Supporting Innovative Teaching. In: JRC-IPTS (2009).
- [2] NACCCE: All Our Futures: Creativity, Culture and Education (1999).
- [3] Ellis, S., Barrs, M.: The Assessment of Creative Learning. In: Sefton-Green, J. (ed.) Creative Learning, pp. 73–89. Creative Partnerships, London (2008).
- [4] Cachia, R., Ferrari, A., Ala-Mutka, K., Punie, Y.: Creative Learning and Innovative Teaching: Final Report on the Study on Creativity and Innovation in Education in EU Member States. In: JRC-IPTS (2010).
- [5] <https://moodle2.tu-ilmenau.de/> , retrieved on 13-01-2019.
- [6] K. Henke, K. Debes, H.-D. Wuttke and A. Katzmann, “Mobile Assessment Tool”, International Conference on Interactive Computer-Aided Blended Learning. ICBL., Florianapolis, Brazil, 2013.
- [7] Hamann, Ch. Saul and H-D. Wuttke “PANDA: A Platform for Open Learning Analytics”, 7th CSEDU 2015: Lisbon, Portugal.
- [8] Buckingham Shum, Simon and Deakin Crick, Ruth (2012). Learning dispositions and transferable competencies: pedagogy, modelling and learning analytics. In: 2nd International Conference on Learning Analytics & Knowledge, 29 Apr - 02 May 2012, Vancouver, British Columbia, Canada.
- [9] <http://www.cs.waikato.ac.nz/ml/weka/> retrieved on 13-01-2019.
- [10] <https://deeplearning4j.org/> retrieved on 13-01-2019.
- [11] Quinlan, J. R. C4.5: Programs for Machine Learning. Morgan Kaufmann Publishers, 1993.
- [12] L. Breiman, J. H. Friedman, R. A. Olshen, C. J. Stone: CART: Classification and Regression Trees. Wadsworth: Belmont, CA, (1984).
- [13] The Grid of Online Laboratory Devices Ilmenau (GOLDi), <http://www.goldi-labs.net/> retrieved on 13-01-2019.
- [14] Thai-Nghe, Nguyen, “Predicting Student Performance in an Intelligent Tutoring System”, PhD thesis, Department of Computer Science Information Systems and Machine Learning Lab (ISMLL) University of Hildesheim, Germany.
- [15] Dirk T. Tempelaar, Bart Rienties and Bas Giesbers, “Stability and sensitivity of learning Analytics based prediction Models”. 7th CSEDU 2015: Lisbon, Portugal.
- [16] S. Huang, “Predictive Modeling and Analysis of Student Academic Performance in an Engineering Dynamics Course”. PhD thesis, Utah State University 12-201.